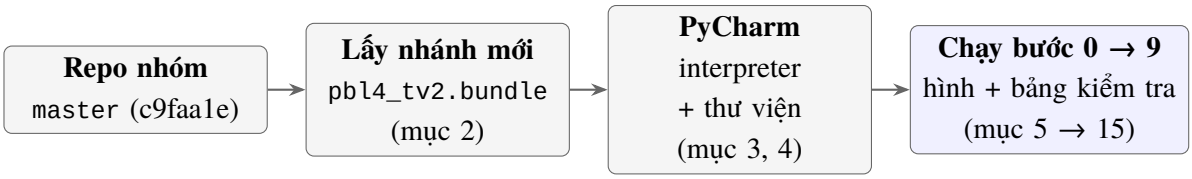
HƯỚNG DẪN CHẠY DỰ ÁN TRONG PYCHARM

DM + GNN định tuyến SAGSIN, viết tiếp từ repo của nhóm, từng tệp, có test và hình minh họa

TV2 · nhánh `tv2-dm-gnn` · tháng 9/2026

Tài liệu này bắt đầu từ đúng trạng thái hiện tại của repo nhóm trên GitHub: nhánh `master`, commit `c9faa1e` (*add_gnn*, TV1 đẩy lên ngày 25/9/2026). Mọi phần viết thêm nằm trên một nhánh riêng tên `tv2-dm-gnn`, chưa đẩy lên GitHub. Bạn nhận nhánh này dưới dạng một tệp `pbl4_tv2.bundle` (hoặc tệp nén `PBL4_tv2_day_du.zip` nếu không muốn dùng git), mở trong PyCharm và chạy mười bước. Mỗi bước là một tệp Python trong thư mục `huong_dan`; chạy xong, bước đó vẽ hình trong PyCharm và kết thúc bằng một bảng kiểm tra xanh / đỏ. Terminal chỉ in một dòng tóm tắt.

Tất cả đã được chạy thử trên hai môi trường: Python 3.11 với torch 2.14, và Python 3.13 với torch 2.6.0, numpy 2.5, skyfield 1.49 (gần với máy của nhóm, vì cấu hình PyCharm trong repo ghi *Python 3.13 (PBL4)*). Kết quả: 97/97 kiểm tra của mười bước đạt, 49/49 test của pytest đạt. Các hình trong tài liệu là kết quả thật của lần chạy đó. Riêng các hình giao diện PyCharm là hình vẽ giản lược để chỉ chỗ bấm, không phải ảnh chụp màn hình; giao diện thật có thể khác đôi chút tùy phiên bản.



1. Nhánh mới có gì so với master

Nhánh `tv2-dm-gnn` giữ nguyên mọi tệp của nhóm và chỉ thêm tệp mới, trừ ba tệp được sửa nhỏ (dòng cuối bảng). Các sửa đổi đó chỉ thêm tham số tùy chọn, nên mã cũ của nhóm vẫn chạy như trước.

Tệp / thư mục	Nội dung
`huong_dan/buoc_0...9_*.py`	Mười bước hướng dẫn, mỗi bước một tệp chạy độc lập.
`huong_dan/chay_tat_ca.py`	Chạy cả mười bước, mở trang kết quả tổng hợp trong trình duyệt.
`huong_dan/tien_ich.py`	Hàm vẽ mạng theo tầng, bảng kiểm tra xanh / đỏ, gọi mô hình.
`huong_dan/du_lieu_demo/`	Dữ liệu demo 73, 173, 273, 373 nút (6 snapshot mỗi quy mô, 880 KB).
`truc_quan_dm.py`	Viết tiếp `visualization.py` của nhóm: thêm tuyến DM, số đo nhiễu, vùng mưa lên bản đồ.
`src/ai/train_da_quy_mo.py`	Huấn luyện một mô hình DM dùng cho mọi quy mô mạng.
`src/ngau_nhien/`	Thời tiết thật (ERA5), kịch bản mưa / giông / tải, nhiễu do mưa theo vật lý.
`test/test_gnn.py`, `test_mo_rong.py`, `test_ngau_nhien.py`	40 test mới (cộng 9 test `test_dataset.py` của TV1 là 49).
`test/so_sanh_*.py`, `luat_chia_doi.py`, …	Các tệp thí nghiệm đầy đủ đã dùng cho báo cáo.

Tệp / thư mục	Nội dung
`results/dmx0_4lop.pt`, `results/quy_mo/*.pt`	Mô hình đã huấn luyện, mỗi tệp khoảng 0,4 MB.
`data/thoi_tiet_vn.npz`, `data/topojson/`	Mưa, giông từng giờ 2024–2025 tại 23 điểm; bản đồ nền để bản đồ chạy không cần mạng.
`tai_lieu/*.pdf`	Các báo cáo: quy mô mạng, cơ sở nhiễu mưa giông, mở rộng ngẫu nhiên, và tài liệu này.
`.run/`, `pytest.ini`, `HUONG_DAN.md`	Cấu hình chạy cho PyCharm, cấu hình pytest, hướng dẫn ngắn.
Sửa: `src/ai/data_v2.py`, `src/ai/train_gnn.py`, `.gitignore`	Thêm tham số tùy chọn (nhiễu đo khi huấn luyện, đặc trưng nút theo snapshot); cho phép các tệp `.pt` nhỏ ở trên vào git và bỏ qua thư mục kết quả `huong_dan/ket_qua/`.

2. Lấy nhánh mới về máy

Cách 1, dùng git (nên dùng). Chép tệp `pbl4_tv2.bundle` vào một thư mục bất kỳ, ví dụ `D:\Tai_ve`. Tệp bundle là một "repo thu nhỏ" chứa phần mới của nhánh, nên git lấy nhánh từ tệp này giống như lấy từ GitHub. Phần mới được xây trên commit `c9faa1e` của master, nên repo của bạn phải có commit này: mở repo PBL4 trong PyCharm, bấm **Git → Pull** (hoặc gõ `git pull origin master`) nếu chưa cập nhật. Sau đó mở Terminal (**Alt+F12**) và gõ hai lệnh:

```
git fetch D:\Tai_ve\pbl4_tv2.bundle tv2-dm-gnn:tv2-dm-gnn
git checkout tv2-dm-gnn
```

Lệnh đầu tạo nhánh `tv2-dm-gnn` trong repo của bạn, lệnh sau chuyển sang nhánh đó. Nhánh `master` không bị đụng tới; muốn quay lại, gõ `git checkout master` hoặc bấm vào tên nhánh ở góc trên bên trái PyCharm. Nếu máy bạn chưa có repo, trước hết gõ `git clone https://github.com/huyth1204/PBL4.git` rồi mở thư mục vừa tải trong PyCharm.

Cách 2, không dùng git. Giải nén `PBL4_tv2_day_du.zip`: đó là toàn bộ dự án ở trạng thái của nhánh mới (không kèm lịch sử git). Mở thư mục đã giải nén trong PyCharm và làm tiếp từ mục 3.



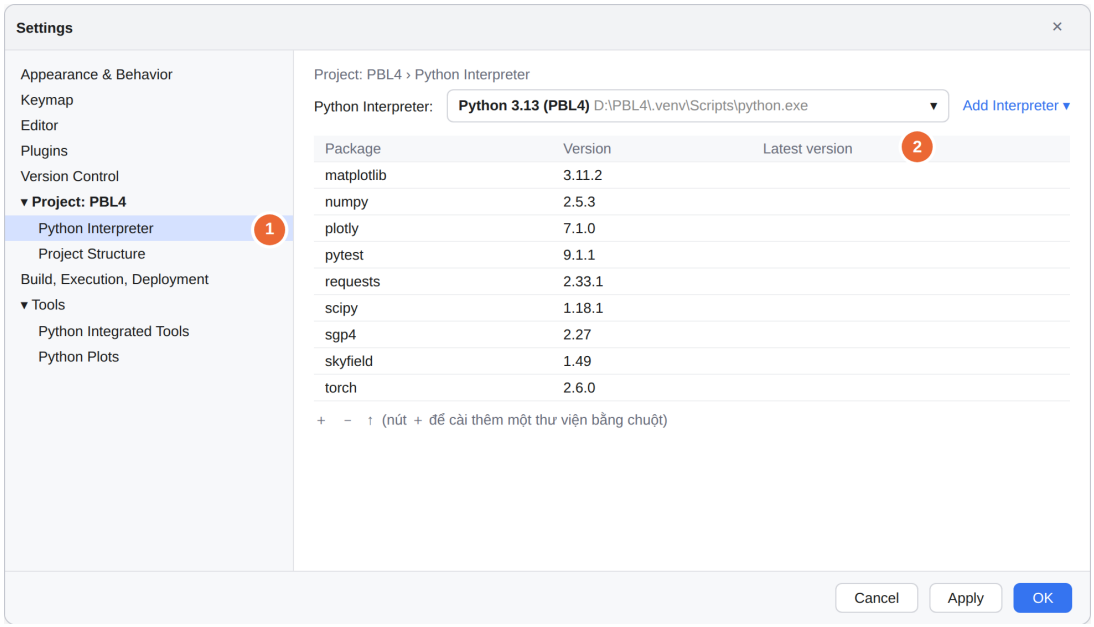
Hình 1. Terminal trong PyCharm (hình giản lược): ① lấy nhánh từ tệp bundle, ② chuyển sang nhánh mới, ③ cài thư viện (mục 4). Chữ `(.venv)` ở đầu dòng cho biết Terminal đang dùng đúng môi trường Python của dự án.

3. Mở dự án và chọn interpreter

Mở dự án. **File → Open**, chọn thư mục PBL4 (thư mục có `src`, `dataset`, `test`), bấm **Trust Project** nếu PyCharm hỏi. Repo của nhóm có sẵn thư mục `.idea` (cấu hình PyCharm của TV1), nên dự án

mở ra đã có tên PBL4 và thư mục `src` được đánh dấu là thư mục mã nguồn.

Chọn interpreter. Cấu hình của TV1 trỏ tới `D:\PBL4\.venv`. Nếu máy bạn có môi trường ở chỗ khác, PyCharm sẽ báo *Invalid Python interpreter* ở góc dưới bên phải. Sửa như sau: **Ctrl+Alt+S** (Settings) → **Project: PBL4** → **Python Interpreter** → **Add Interpreter** → **Add Local Interpreter** → **Virtualenv Environment** → **New**, chọn Python 3.10 đến 3.13 làm *Base interpreter*, bấm OK (bản PyCharm mới ghi là *Generate new* thay cho *New*, *Select existing* thay cho *Existing*). PyCharm tạo thư mục `.venv` trong dự án và dùng nó cho mọi lần chạy. Nếu bạn đã có môi trường riêng cho PBL4 thì chọn **Existing** và trỏ tới `python.exe` trong đó.



Hình 2. Settings → Project: PBL4 → Python Interpreter (hình giản lược): ① mục cần mở, ② ô chọn interpreter; bảng dưới liệt kê thư viện đã cài. Phiên bản trong hình là phiên bản đã chạy thử.

4. Cài thư viện

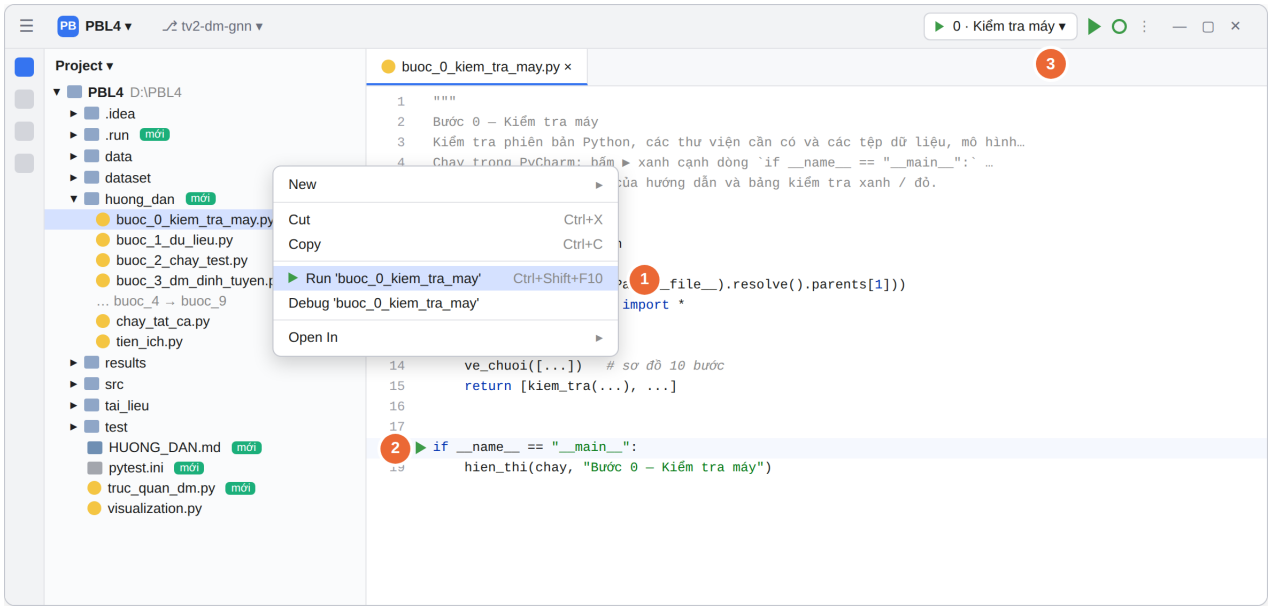
Trong Terminal của PyCharm (**Alt+F12**), kiểm tra đầu dòng có `(.venv)` rồi gõ:

```
pip install -r huong_dan/yeu_cau.txt
```

Tệp `huong_dan/yeu_cau.txt` gồm numpy, scipy, torch, matplotlib, plotly, skyfield, sgp4, requests, pytest. Nếu máy đã cài theo `requirements.txt` của nhóm thì lệnh này chỉ cài thêm những gì còn thiếu (thường là matplotlib, plotly, pytest). Torch là gói lớn nhất (vài trăm MB), chỉ tải một lần. Không cần mạng cho phần còn lại: dữ liệu thời tiết, bản đồ nền và mô hình đều có sẵn trong nhánh.

Cách bằng chuột: trong Hình 2, bấm dấu **+** dưới bảng thư viện, gõ tên gói, bấm **Install Package**. Cách này cài từng gói một nên chậm hơn.

5. Ba cách chạy một bước

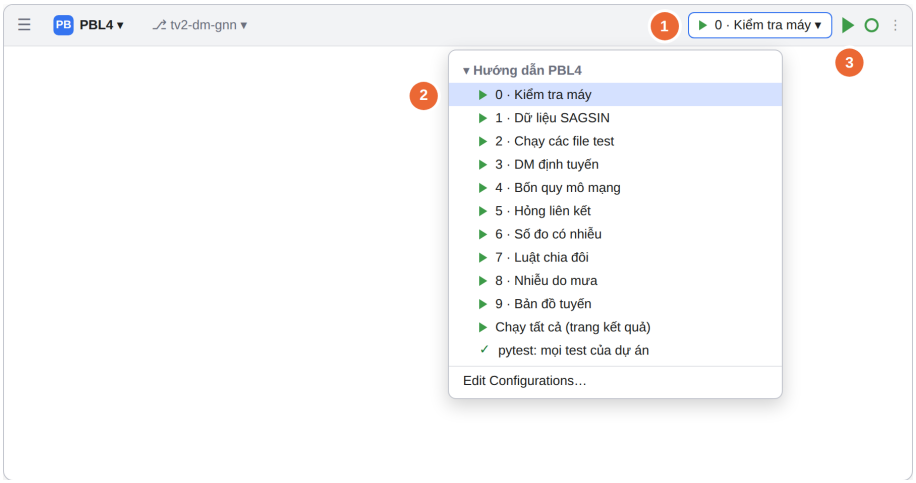


Hình 3. Ba cách chạy tệp `buoc_0_kiem_tra_may.py` (hình giản lược). Các tệp và thư mục có nhãn "mới" là phần của nhánh `tv2-dm-gnn`.

Cách ①. Trong cây thư mục bên trái, chuột phải vào tệp → **Run 'buoc_0_kiem_tra_may'** (phím tắt **Ctrl+Shift+F10** khi đang mở tệp).

Cách ②. Mở tệp, bấm tam giác xanh ▶ ở lề trái, cạnh dòng `if __name__ == "__main__":` → **Run**.

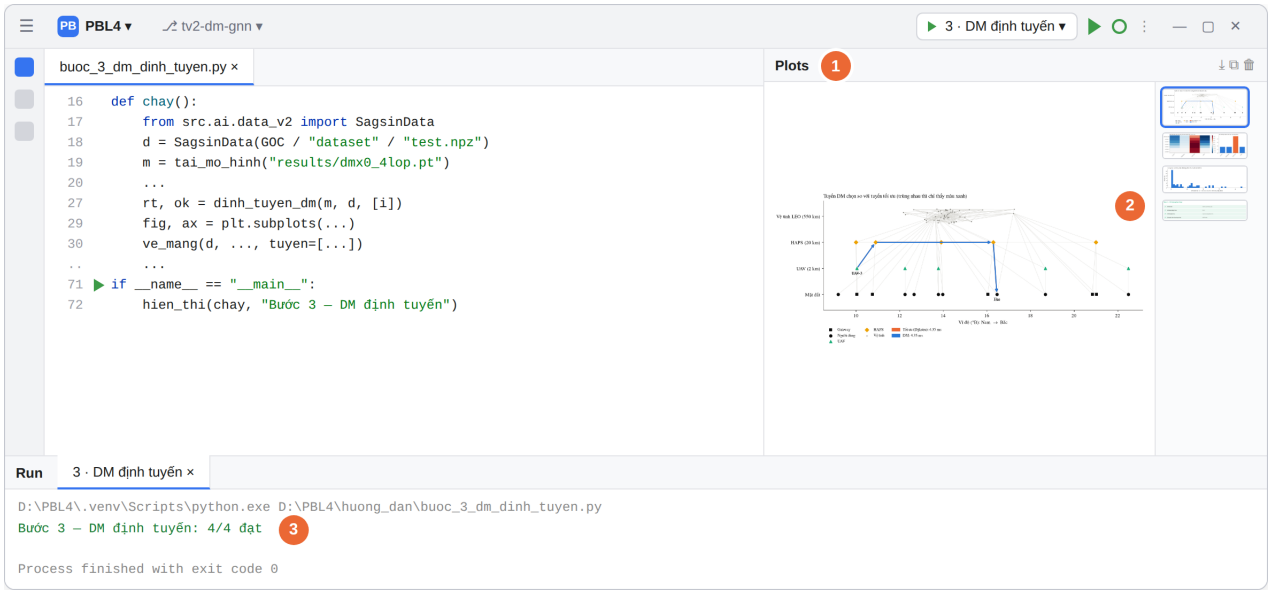
Cách ③, gọn nhất. Dùng ô chọn cấu hình ở góc trên bên phải. Nhánh mới có thư mục `.run` chứa 12 cấu hình dựng sẵn; PyCharm tự đọc thư mục này khi mở dự án và gom chúng vào nhóm *Hướng dẫn PBL4* (Hình 4). Chọn cấu hình rồi bấm ▶. Các cấu hình này đặt sẵn thư mục làm việc là thư mục gốc dự án; thực ra các tệp bước tự tìm thư mục gốc nên chạy bằng cách nào cũng được.



Hình 4. Ô chọn cấu hình (hình giản lược): ① bấm vào ô, ② chọn bước, ③ bấm ▶. Nếu không thấy nhóm *Hướng dẫn PBL4*, đóng rồi mở lại dự án, hoặc dùng cách 1.

Kết quả hiện ở đâu. Mỗi bước vẽ vài hình rồi thêm hình cuối là bảng kiểm tra. Nếu PyCharm của bạn có tab **Plots** (mặc định bật; kiểm tra ở **Settings → Tools → Python Plots → Show plots in tool window**, bản cũ đặt ở **Tools → Python Scientific**), mọi hình hiện ở tab Plots bên phải, cột ảnh nhỏ

bên cạnh để chuyển hình (Hình 5). Nếu không có tab Plots, mỗi hình mở thành một cửa sổ riêng; xem xong, đóng hết các cửa sổ thì lần chạy mới kết thúc. Cửa sổ **Run** ở dưới chỉ in một dòng, ví dụ *Bước 3 — DM định tuyến: 4/4 đạt*.



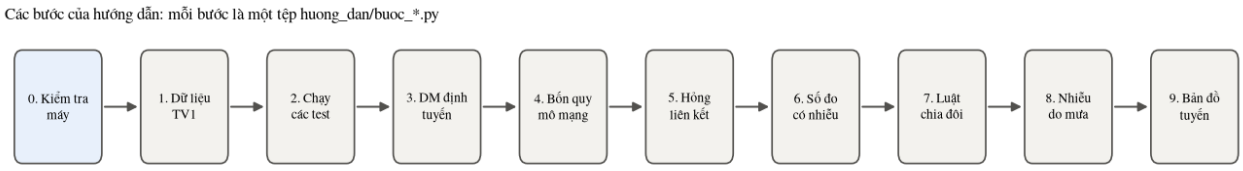
Hình 5. Sau khi chạy bước 3 (hình giản lược, hình vẽ bên trong là kết quả thật): ① tab Plots, ② ảnh nhỏ của bốn hình, hình cuối là bảng kiểm tra, ③ dòng tóm tắt trong cửa sổ Run.

Đọc bảng kiểm tra: hàng xanh có dấu ✔ là đạt; hàng đỏ có dấu ✘ là không đạt, cột giữa ghi lý do (con số đo được, hoặc tên lỗi Python). Mỗi kiểm tra là một điều phải đúng nếu dự án chạy đúng, ví dụ "tuyến DM chỉ dùng liên kết còn sống". Một bước bị lỗi Python giữa chừng cũng không làm dừng các kiểm tra khác: lỗi được ghi thành một hàng đỏ.

6. Bước 0 — Kiểm tra máy

Tệp chạy: `huong_dan/buoc_0_kiem_tra_may.py` · **Cấu hình:** *0 · Kiểm tra máy* · **Thời gian:** vài giây

Chạy bước này đầu tiên. Nó kiểm tra phiên bản Python (cần 3.10 trở lên), chín thư viện, và tám tệp dữ liệu / mô hình mà các bước sau dùng tới. Hình đầu là sơ đồ mười bước; hình thứ hai là bảng 17 kiểm tra, mỗi thư viện ghi phiên bản đang cài và bước nào cần nó.



Bước 0 — Kiểm tra máy: 17/17 đạt

✔ Python ≥ 3.10	3.13.13
✔ Thư viện numpy	phiên bản 2.5.3 — dùng cho mọi bước
✔ Thư viện scipy	phiên bản 1.18.1 — dùng cho mọi bước
✔ Thư viện torch	phiên bản 2.6.0+cu124 — dùng cho mô hình DM
✔ Thư viện matplotlib	phiên bản 3.11.2 — dùng cho vẽ hình
✔ Thư viện plotly	phiên bản 7.1.0 — dùng cho bản đồ, bước 9
✔ Thư viện skyfield	phiên bản 1.49 — dùng cho vị trí vệ tinh, bước 9
✔ Thư viện requests	phiên bản 2.34.2 — dùng cho một test thời tiết
✔ Thư viện pytest	phiên bản 9.1.1 — dùng cho chạy test trong PyCharm
✔ Có tệp dataset/test.npz	
✔ Có tệp dataset/oracle_dataset_v2.npz	
✔ Có tệp data/tle/starlink.txt	
✔ Có tệp data/thoi_tiet_vn.npz	
✔ Có tệp results/dmx0_4lop.pt	
✔ Có tệp results/quy_mo/dm_da_quy_mo.pt	
✔ Có tệp results/quy_mo/dm_nhieu.pt	
✔ Có tệp huong_dan/du_lieu_demo/n373.npz	

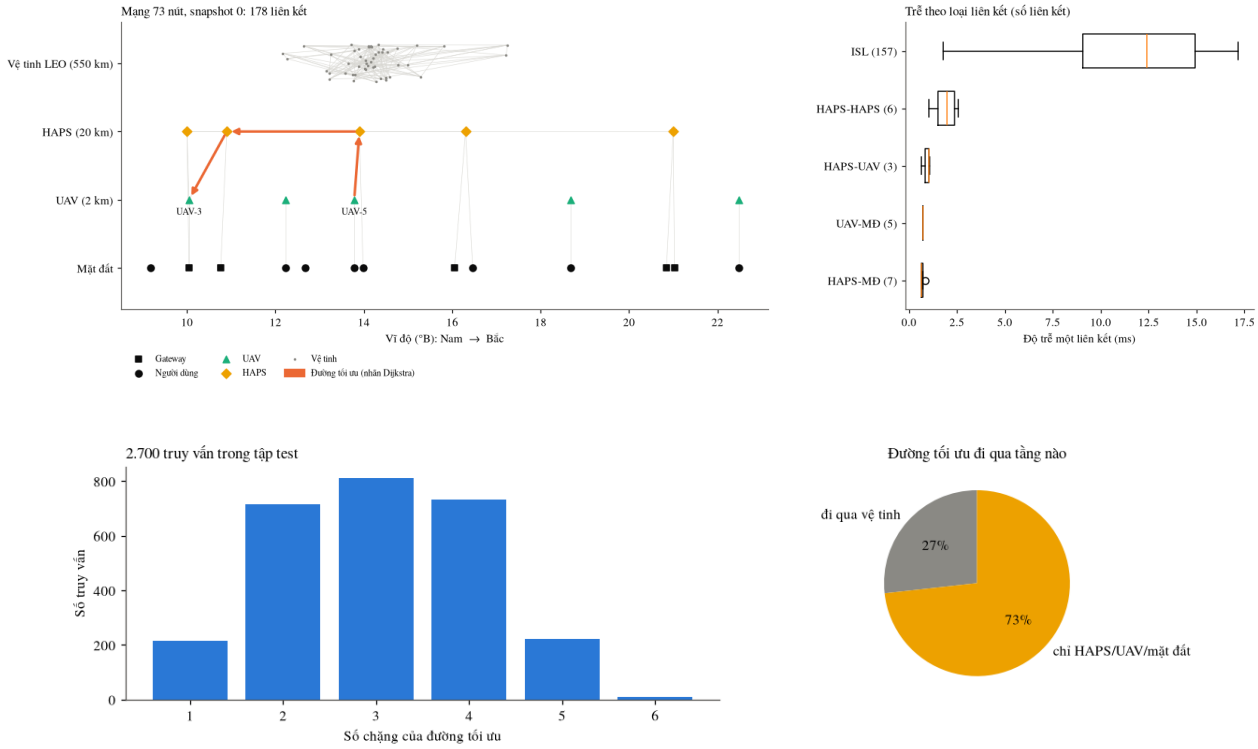
Nếu có hàng đỏ. Thư viện đỏ: chạy lại lệnh cài ở mục 4, rồi kiểm tra interpreter đúng là môi trường vừa cài (Hình 2). Tệp đỏ: bạn đang ở nhánh `master` chứ không phải `tv2-dm-gnn` (xem tên nhánh ở góc trên bên trái), hoặc giải nén thiếu. Riêng `dataset/oracle_dataset_v2.npz` là tệp của TV1 có sẵn trên master.

7. Bước 1 — Dữ liệu SAGSIN của TV1

Tệp chạy: `huong_dan/buoc_1_du_lieu.py` · **Tệp dự án:** `dataset/oracle_labeler_sagsin.py` (sinh dữ liệu), `dataset/split_dataset.py` (chia), `src/ai/data_v2.py` (đọc) · **Dữ liệu:** `dataset/test.npz`

Bước này xem dữ liệu mà cả dự án dựa vào. Tập test có 54 snapshot và 2.700 truy vấn trên mạng 73 nút: 50 vệ tinh LEO, 5 HAPS, 5 UAV, 5 gateway, 8 người dùng. Mỗi snapshot là một ma trận trễ W: W_{ij} là độ trễ (ms) của liên kết $i \to j$, bằng -1 nếu không có liên kết. Mỗi truy vấn là một cặp nguồn–đích kèm nhãn là đường có tổng trễ nhỏ nhất do Dijkstra tìm.

Hình đầu vẽ mạng theo tầng: trục đứng là độ cao (mặt đất, UAV 2 km, HAPS 20 km, vệ tinh 550 km), trục ngang là vĩ độ thật của nút từ Nam ra Bắc; mũi tên cam là một đường tối ưu. Bên phải là độ trễ theo loại liên kết: liên kết giữa hai vệ tinh (ISL) dài vài nghìn km nên trễ 2–17 ms, còn liên kết HAPS / UAV / mặt đất chỉ dưới 3 ms. Hình thứ hai cho biết đường tối ưu thường dài 2–4 chặng, và 73% đường không cần đi qua vệ tinh.



Bước 1 — Dữ liệu TV1: 4/4 đạt

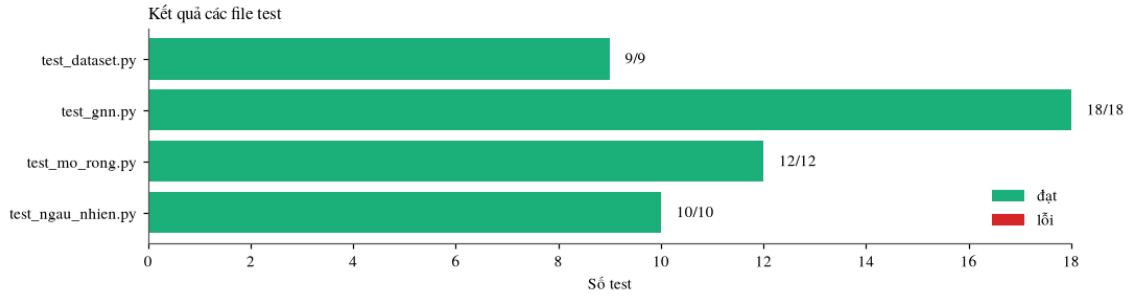
✔ Ma trận trễ đối xứng	W[i,j] = W[j,i]
✔ Nhãn đúng là đường Dijkstra	270 truy vấn, 0 truy vấn lệch
✔ Người dùng không làm trạm trung gian	2700 đường
✔ Mọi nhãn đi trên liên kết có thật	

Bốn kiểm tra: ma trận trễ đối xứng; chạy lại Dijkstra trên 270 truy vấn thì trùng nhãn cả 270; người dùng không bao giờ làm trạm trung gian (quy tắc chuyển tiếp của TV1); mọi chặng trong nhãn là liên kết có thật.

8. Bước 2 — Chạy các file test

Tệp chạy: `huong_dan/buoc_2_chay_test.py` · **Tệp test:** `test/test_dataset.py` (9, của TV1), `test/test_gnn.py` (18), `test/test_mo_rong.py` (12), `test/test_ngau_nhien.py` (10) · **Thời gian:** 10–60 giây

Bước này gọi lần lượt mọi hàm `test_*` trong bốn tệp test và vẽ số test đạt theo tệp. Mỗi test kiểm một tính chất cụ thể, ví dụ: mô hình không bao giờ chọn nút bị cấm; kết quả không đổi khi đánh số lại các nút; nhiễu do mưa bằng 0 khi trời không mưa; mưa càng to và tần số càng cao thì nhiễu càng lớn. Một test (`test_tai_open_meteo_va_them_giong`) giả lập việc tải thời tiết từ Internet nên không cần mạng.

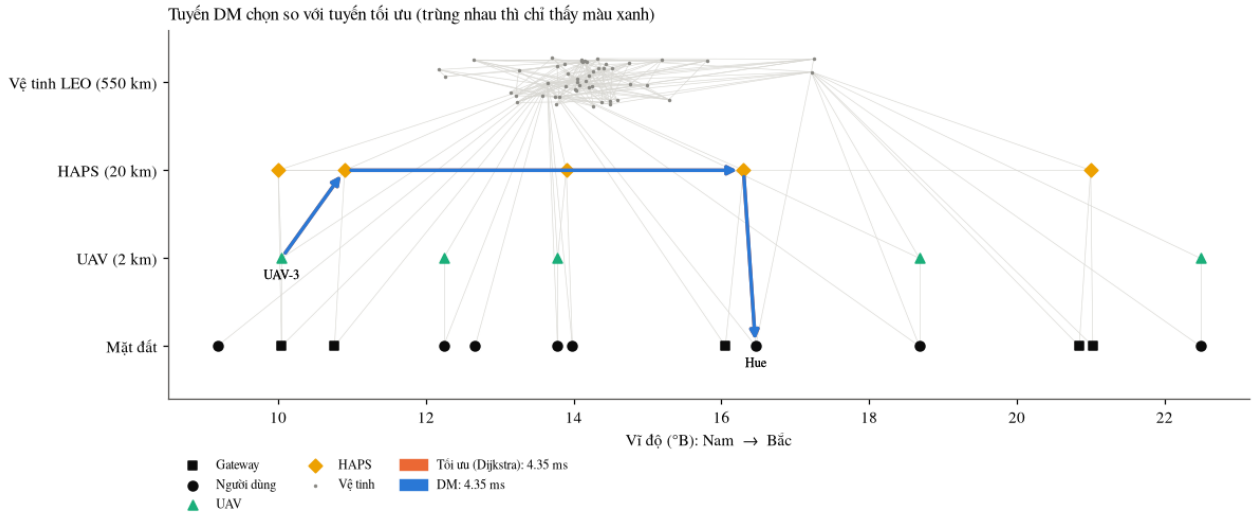


Cách thứ hai, đồ họa hơn, là pytest của PyCharm (mục 16): cây test có dấu ✔ xanh cho từng hàm, bấm vào một test đỏ để xem dòng gây lỗi.

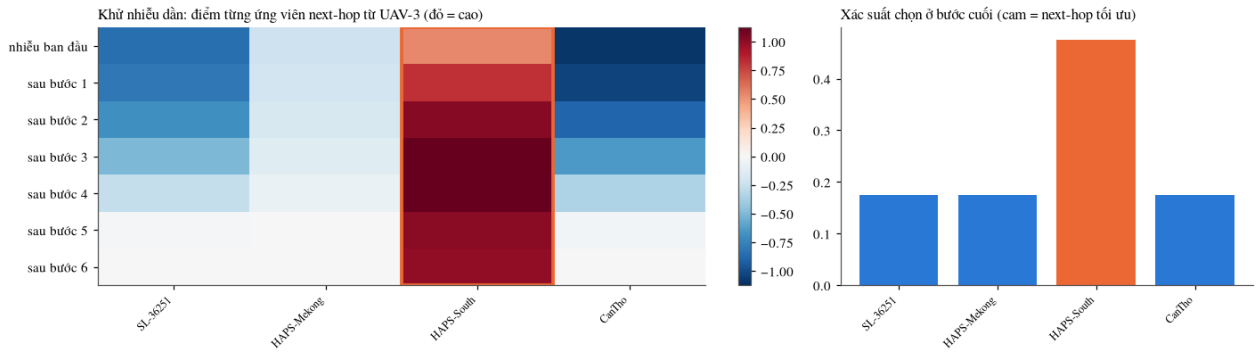
9. Bước 3 — Mô hình GNN + DM định tuyến

> **Tệp chạy:** `huong_dan/buoc_3_dm_dinh_tuyen.py` · **Tệp dự án:** `src/ai/gnn_diffusion_v2.py` (mô hình), `src/ai/train_gnn.py` (huấn luyện, đi từng bước) · **Mô hình:** `results/dmx0_4lop.pt` (4 lớp GNN, 107.330 tham số)

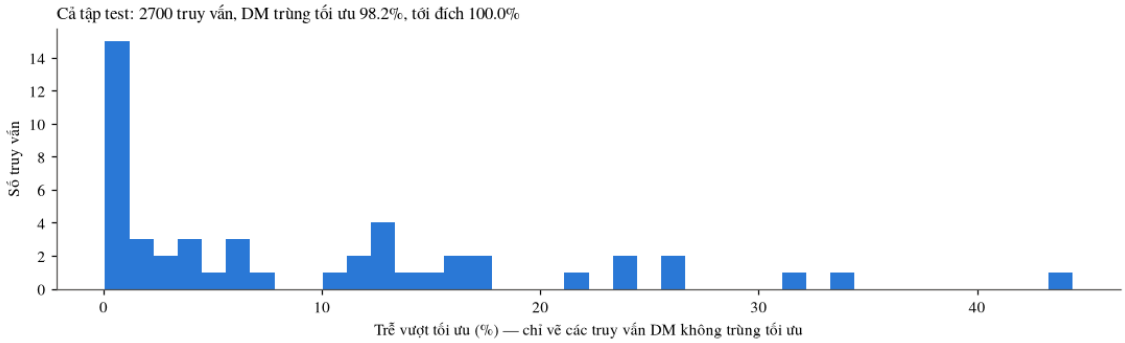
Mô hình chọn đường theo từng chặng. Ở nút hiện tại, GNN đọc cả mạng (trễ các liên kết, loại nút, nút đích) và tóm mỗi nút thành một vectơ 64 số. Phần khuếch tán (DM) bắt đầu từ một dãy điểm số ngẫu nhiên cho các nút kề có thể đi tới (các "ứng viên next-hop"), rồi khử nhiễu dần: mỗi bước, mạng nơ-ron đoán điểm số sạch rồi trộn lại với phần nhiễu còn lại, nhiễu giảm dần tới 0. Ứng viên có điểm cao nhất được chọn, mô hình sang nút đó và lặp lại tới đích. Khi dùng thật chỉ cần 2 bước khử nhiễu; hình thứ hai vẽ 6 bước để dễ nhìn.



Hình trên: truy vấn từ UAV-3 tới Huế. Tuyến DM (xanh) trùng tuyến tối ưu nên che mất màu cam; cả hai đều 4,35 ms.



Hình trên: mỗi hàng là điểm số của bốn ứng viên sau một bước khử nhiễu (đỏ = cao, xanh = thấp). Ban đầu điểm số lộn xộn; sau vài bước, ứng viên HAPS-South (khung cam, là next-hop tối ưu) nổi hẳn lên. Cột bên phải là xác suất chọn ở bước cuối.



Hình trên: trên cả 2.700 truy vấn, DM tới đích 100% và trùng tuyến tối ưu 98,2%. Biểu đồ chỉ vẽ 1,8% truy vấn không trùng (49 truy vấn): hơn một nửa trong số đó chỉ dài hơn tối ưu dưới 8%, số còn lại 10–45%. Tính trung bình trên mọi truy vấn, trễ DM vượt tối ưu 0,18%.

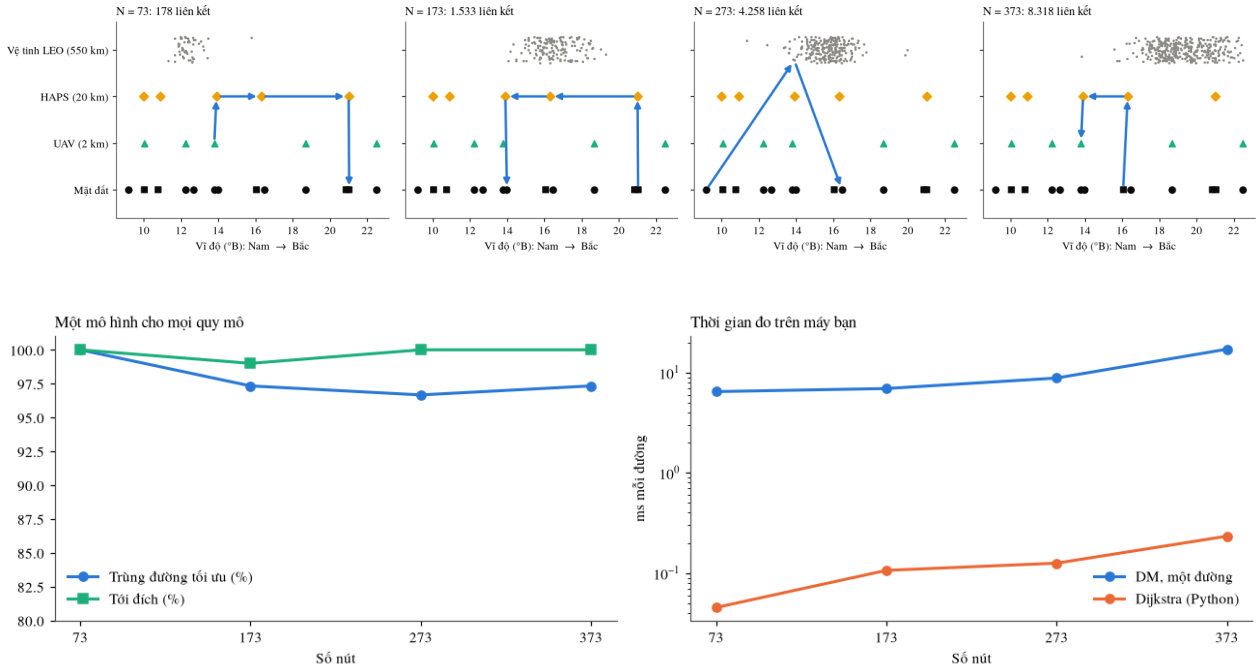
Bước 3 — DM định tuyến: 4/4 đạt

✔ DM tới đích	100.0% của 2700 truy vấn
✔ DM trùng đường tối ưu	98.2%
✔ Trễ DM gần tối ưu	vượt tối ưu trung bình 0.18%
✔ Khử nhiễu dồn về next-hop tối ưu	HAPS-South

10. Bước 4 — Bốn quy mô mạng

Tệp chạy: `huong_dan/buoc_4_quy_mo.py` · **Tệp dự án:** `test/danh_gia_quy_mo.py`, `src/ai/train_da_quy_mo.py` · **Mô hình:** `results/quy_mo/dm_da_quy_mo.pt` · **Tài liệu:** `tai_lieu/DM_theo_quy_mo_mang.pdf`

Mạng được làm lớn dần bằng cách thêm vệ tinh: 73 → 173 → 273 → 373 nút (dùng chính bộ sinh dữ liệu của TV1). Vì GNN xử lý từng nút và từng liên kết giống nhau, cùng một bộ trọng số chạy được trên mọi kích thước mạng. Bước này chạy một mô hình duy nhất trên dữ liệu demo của cả bốn quy mô, rồi đo thời gian ngay trên máy bạn.



Bước 4 — Quy mô mạng: 4/4 đạt

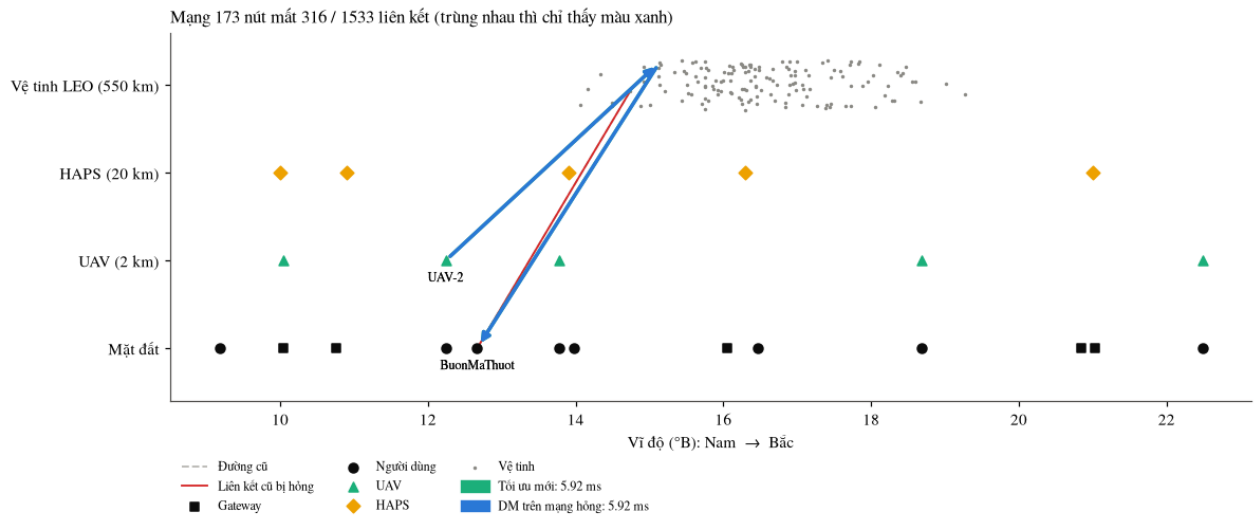
✔	N = 73	tới đích 100.0%, trùng 100.0%, vượt 0.00%, DM 6.5 ms / Dijkstra 0.05 ms
✔	N = 173	tới đích 99.0%, trùng 97.3%, vượt 0.06%, DM 6.9 ms / Dijkstra 0.11 ms
✔	N = 273	tới đích 100.0%, trùng 96.7%, vượt 0.10%, DM 8.8 ms / Dijkstra 0.13 ms
✔	N = 373	tới đích 100.0%, trùng 97.3%, vượt 0.11%, DM 17.1 ms / Dijkstra 0.23 ms

Đọc kết quả: DM trùng tối ưu 96–100% và tới đích 99–100% ở mọi quy mô, trễ vượt tối ưu chỉ 0–0,11%. Về thời gian, Dijkstra (dưới 1 ms) luôn nhanh hơn DM (khoảng 5–20 ms một đường, tùy máy). Đây là điều đã biết và ghi rõ trong báo cáo: trên dữ liệu sạch, Dijkstra vừa đúng tuyệt đối vừa nhanh hơn; ưu thế của DM nằm ở các bước 5–7, khi số đo không còn đáng tin. Số thời gian đổi theo từng lần chạy, nên kiểm tra chỉ xét độ chính xác.

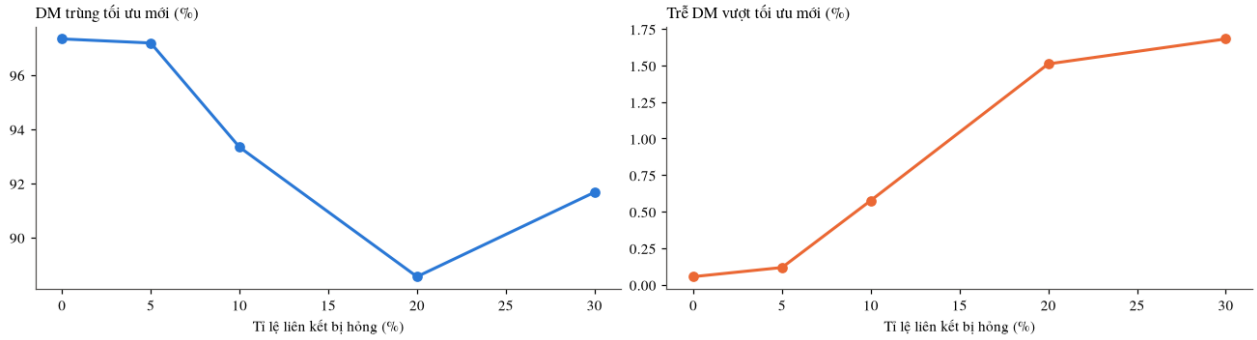
11. Bước 5 — Liên kết bị hỏng

Tệp chạy: `huong_dan/buoc_5_hong_lien_ket.py` · **Tệp dự án:** `test/danh_gia_quy_mo.py` · **Mô hình:** `results/quy_mo/dm_da_quy_mo.pt`

Bỏ ngẫu nhiên 20% liên kết của một snapshot 173 nút (316 trên 1.533 liên kết), chọn một truy vấn mà đường tối ưu cũ đi qua liên kết bị hỏng, rồi cho DM tìm đường trên mạng đã hỏng. DM chưa từng thấy mạng hỏng khi học; nó chỉ biết liên kết nào còn trong ma trận trễ. Sau đó lặp lại với các mức hỏng 0, 5, 10, 20, 30%.



Hình trên: nét đứt xám là đường cũ, đoạn đỏ là liên kết cũ bị hỏng; DM (xanh) tìm được đường mới trùng tối ưu mới (5,92 ms).



Bước 5 — Hỏng liên kết: 4/4 đạt

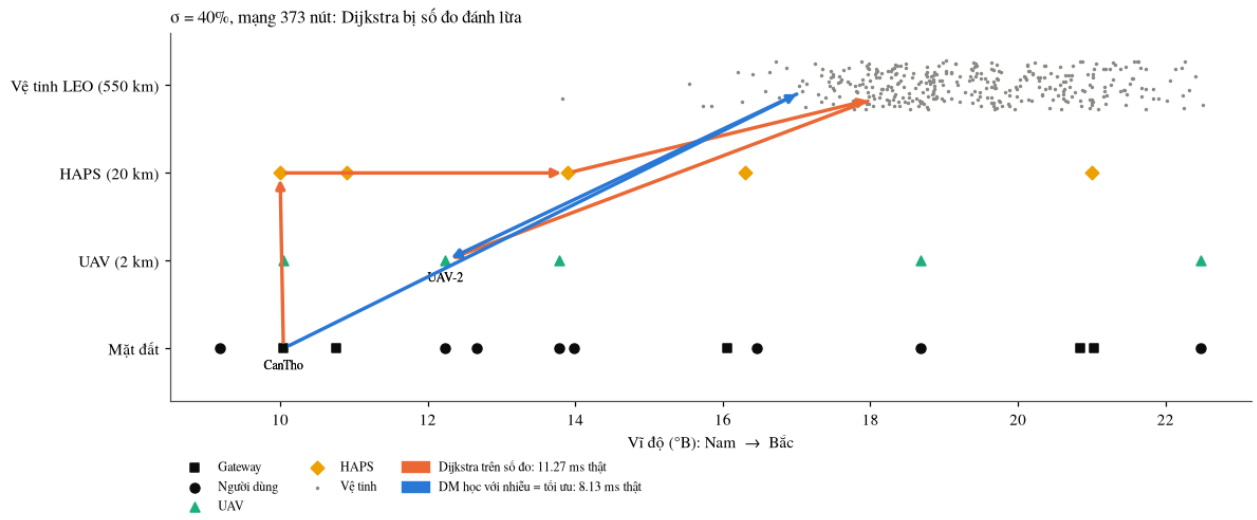
✔ DM tới đích trên mạng hỏng	
✔ Tuyến DM chỉ dùng liên kết còn sống	
✔ Trễ DM so với tối ưu mới	1.000 lần
✔ Hỏng 30%: DM vẫn gần tối ưu mới	vượt 1.68%, trùng 91.7%

Ngay cả khi hỏng 30% liên kết, DM vẫn trùng tối ưu mới hơn 90% và trễ chỉ vượt 1,7%. Đường cong trùng tối ưu không giảm đều (20% thấp hơn 30%) vì mỗi mức hỏng là một lần bốc ngẫu nhiên khác nhau trên 150 truy vấn; với dữ liệu demo nhỏ, dao động vài phần trăm là bình thường.

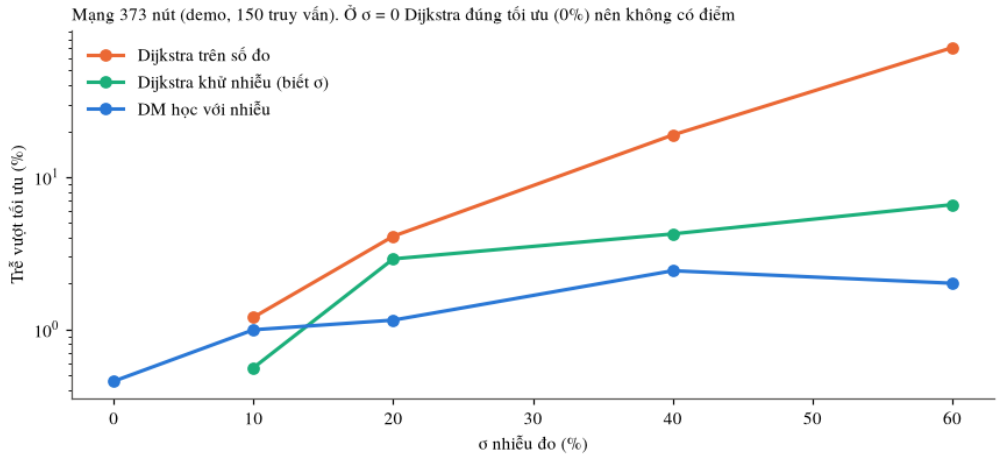
12. Bước 6 — So với Dijkstra khi số đo có nhiễu

Tệp chạy: `huong_dan/buoc_6_nhieu_do.py` · **Tệp dự án:** `test/so_sanh_dijkstra_nhieu.py` · **Mô hình:** `results/quy_mo/dm_nhieu.pt` (học thêm với số đo nhiễu)

Trong thực tế bộ định tuyến không biết trễ thật W, mà chỉ thấy số đo $\tilde{W} = W(1 + \sigma z)$, với z là số ngẫu nhiên chuẩn và σ là độ lệch tương đối của phép đo ($\sigma = 40\%$ nghĩa là số đo thường lệch khoảng 40% so với thật). Dijkstra tin tuyệt đối vào số đo nên hay chọn những liên kết tình cờ bị đo thấp hơn thật. DM được học với số đo nhiễu nên biết "nghi ngờ" số đo, dựa thêm vào cấu trúc mạng. Mọi tuyến đều được chấm bằng trễ thật.



Hình trên: $\sigma = 40\%$, từ Cần Thơ tới UAV-2. Dijkstra trên số đo (cam) đi vòng qua ba HAPS và một vệ tinh, trễ thật 11,27 ms; DM (xanh) đi thẳng qua một vệ tinh, đúng tuyến tối ưu 8,13 ms.



Bước 6 — Số đo có nhiễu: 3/3 đạt

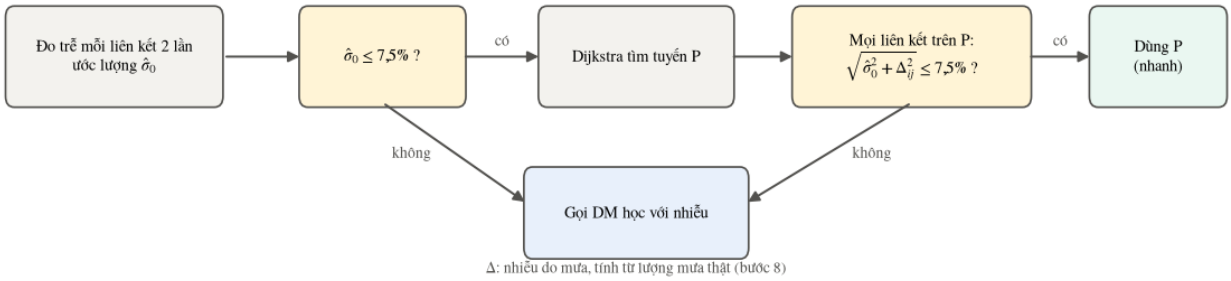
✔	σ = 0: Dijkstra đúng tối ưu	
✔	σ = 40%: DM tốt hơn Dijkstra trên số đo	2.43% so với 18.90%
✔	σ = 40%: DM tốt hơn cả Dijkstra khử nhiễu	2.43% so với 4.24%

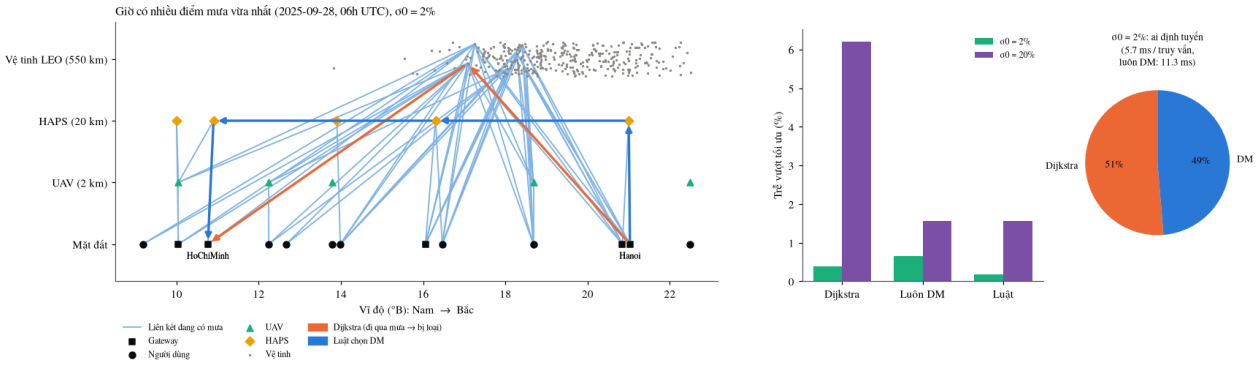
Đọc đồ thị (trục đứng thang log): Dijkstra trên số đo tệ đi rất nhanh theo σ (18,9% ở $\sigma = 40\%$); "Dijkstra khử nhiễu" là Dijkstra được biết trước σ và kéo số đo về trung bình của loại liên kết, tốt hơn nhiều nhưng vẫn kém DM từ $\sigma = 20\%$ trở lên. Ở $\sigma \leq 10\%$ Dijkstra khử nhiễu còn tốt hơn DM: đó là lý do của luật chia đôi ở bước 7. Ở $\sigma = 0$ Dijkstra đúng tối ưu tuyệt đối (0%) nên không có điểm trên thang log.

13. Bước 7 — Luật chia đôi Dijkstra / DM với thời tiết thật

Tệp chạy: `huong_dan/buoc_7_luat_chia_doi.py` · **Tệp dự án:** `test/luat_chia_doi.py`, `src/ngau_nhien/nhieu_mua.py` · **Dữ liệu:** `data/thoi_tiet_vn.npz` (ERA5, 2024–2025)

Hai bước trước cho thấy: số đo tốt thì Dijkstra đúng và nhanh; số đo kém thì DM bền hơn. Luật chia đôi chọn tự động theo sơ đồ dưới. Mỗi liên kết được đo trễ hai lần; hiệu hai lần đo cho biết độ lệch của phép đo $\hat{\sigma}_0$ mà không cần biết trễ thật (lấy trung vị của $|r|$ chia $0{,}6745\sqrt{2}$, với r là hiệu tương đối hai lần đo). Ngoài ra, liên kết đang có mưa bị nhiễu thêm một lượng Δ tính từ lượng mưa thật (bước 8). Nếu $\hat{\sigma}_0 \le 7{,}5\%$ và mọi liên kết trên tuyến Dijkstra có nhiễu dự báo $\sqrt{\hat{\sigma}_0^2 + \Delta^2} \le 7{,}5\%$ thì dùng tuyến Dijkstra; ngược lại gọi DM.





Hình trên: giờ có nhiều điểm mưa vừa nhất trong hai năm dữ liệu (6 giờ UTC ngày 28/9/2025), mạng 373 nút. Nét xanh nhạt là liên kết đang có mưa. Tuyến Dijkstra Hà Nội → TP.HCM (cam) đi qua liên kết có mưa nên bị loại; luật gọi DM (xanh đậm). Cột giữa: trễ vượt tối ưu của ba cách ở $\sigma_0 = 2\%$ (xanh lá) và 20% (tím). Hình tròn: tỉ lệ truy vấn do Dijkstra và DM xử lý ở $\sigma_0 = 2\%$.

Bước 7 — Luật chia đôi: 4/4 đạt

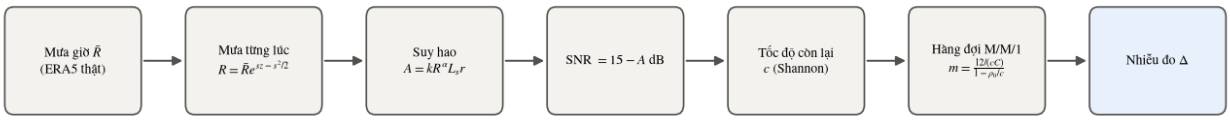
✔ Ước lượng σ0 bằng 2 lần đo	σ0 = 2%: sai 0.0001, σ0 = 20%: sai 0.0016
✔ Số đo tốt: luật không kém luôn DM	0.18% so với 0.65%, gọi DM 49%
✔ Số đo kém: luật giao hết cho DM	gọi DM 100%
✔ Số đo kém: DM tốt hơn Dijkstra	1.57% so với 6.21%

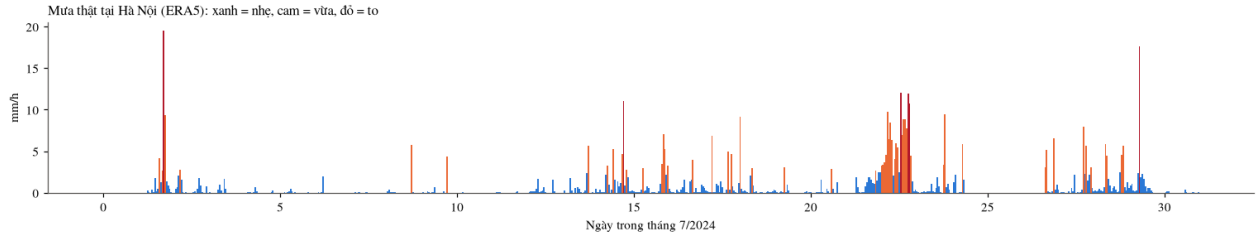
Kết quả: hai lần đo ước lượng σ_0 sai chưa tới 0,2 điểm phần trăm. Khi số đo tốt ($\sigma_0 = 2\%$), luật vượt tối ưu 0,18%, tốt hơn cả luôn dùng DM (0,65%), và chỉ gọi DM cho những truy vấn mà tuyến Dijkstra đi qua mưa; vì đây là giờ mưa nhiều nhất nên tỉ lệ đó cao (khoảng một nửa), ở giờ bình thường chỉ 5–15%. Thời gian trung bình vì thế chỉ bằng khoảng một nửa so với luôn gọi DM. Khi số đo kém ($\sigma_0 = 20\%$), luật giao hết cho DM, và DM (1,57%) tốt hơn hẳn Dijkstra (6,21%).

14. Bước 8 — Nhiễu do mưa và giông, suy từ vật lý

Tệp chạy: `huong_dan/buoc_8_nhieu_mua.py` · **Tệp dự án:** `src/ngau_nhien/nhieu_mua.py` · **Tài liệu:** `tai_lieu/Co_so_nhieu_mua_giong.pdf`

Bước này giải thích con số Δ dùng ở bước 7. Chuỗi suy luận có năm mắt xích, mỗi mắt xích là một công thức quen thuộc: mưa trong giờ dao động quanh trung bình theo phân phối log-chuẩn; giọt mưa hấp thụ và tán xạ sóng nên tín hiệu yếu đi $A = kR^{\alpha}L_s r$ dB (khuyến nghị ITU-R P.838); tín hiệu yếu thì modem hạ tốc độ theo công thức Shannon; tốc độ giảm thì gói tin xếp hàng lâu hơn (hàng đợi M/M/1); thời gian xếp hàng là ngẫu nhiên nên số đo trễ dao động mạnh hơn, và độ dao động tương đối đó chính là Δ.

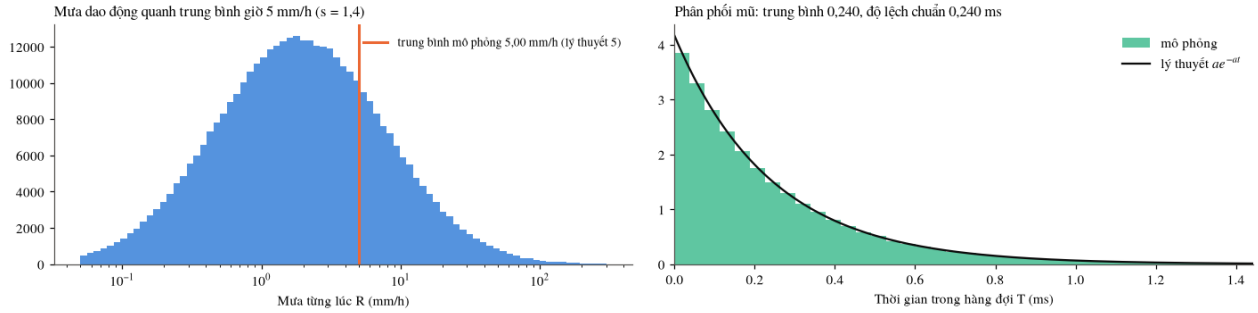




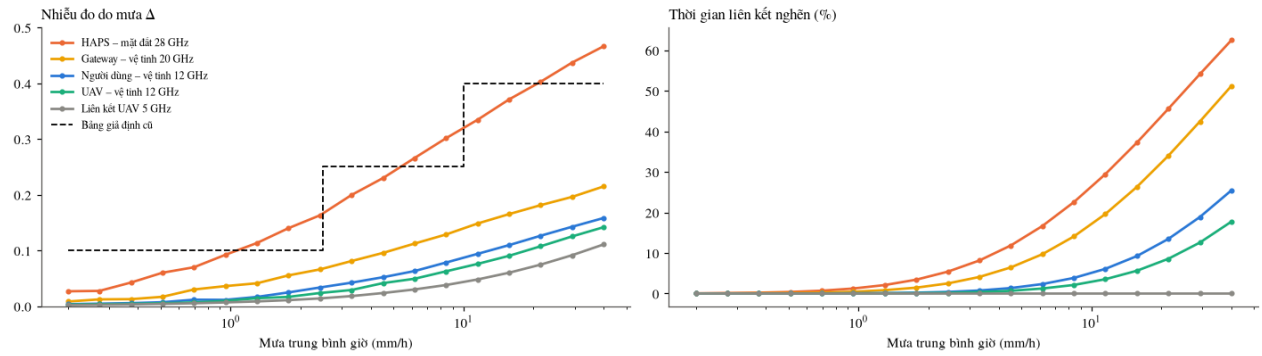
Ví dụ tính tay: gateway – vệ tinh 20 GHz, góc ngẩng 40°, mưa 5 mm/h, W = 3 ms

Bước	Công thức	Giá trị
Quãng đường trong mưa	Ls = 5 / sin 40°	7.78 km
Suy hao do mưa	A = k·R^α·Ls·r (20 GHz)	3.19 dB
Tín hiệu / nhiễu	SNR = 15 − A	11.8 dB
Tốc độ còn lại	c = log2(1+SNR) / log2(1+SNR0)	0.799
Thời gian trong hàng đợi	m0 (trời quang) → m (mưa)	0.171 → 0.241 ms
Nhiễu đo do mưa (tính cả mưa dao động)	Δ = √(2E[m²] − E[m]² − m0²) / W	0.121
Thời gian liên kết nghẽn	P(tải > 90% tốc độ còn lại)	5.2%

Hình trên: ví dụ tính tay cho một liên kết gateway – vệ tinh 20 GHz dưới mưa 5 mm/h. Tín hiệu mất 3,19 dB, tốc độ còn 80%, thời gian xếp hàng tăng từ 0,171 lên 0,241 ms, nhiễu đo $\Delta = 0{,}121$ (12%, đã vượt ngưỡng 7,5% của luật chia đôi).



Hình trên: kiểm chứng hai giả thiết bằng mô phỏng 400.000 mẫu. Trái: mưa từng lúc có trung bình đúng 5 mm/h dù phần lớn thời gian nhỏ hơn 5 (vài trận to kéo trung bình lên). Phải: thời gian xếp hàng có phân phối mũ, độ lệch chuẩn bằng trung bình.



Hình trên: Δ và thời gian nghẽn theo lượng mưa cho từng loại liên kết; nét đứt đen là bảng giả định cũ (trước khi có cơ sở vật lý). Tần số càng cao càng chịu mưa: HAPS – mặt đất 28 GHz tệ nhất, liên kết UAV 5 GHz gần như không bị ảnh hưởng.

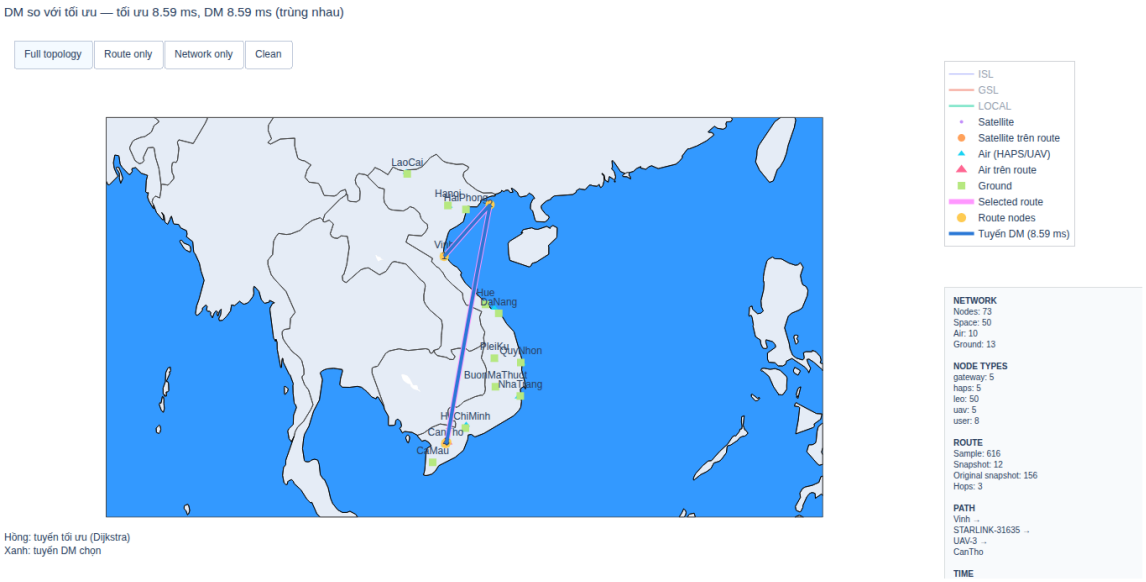
Bước 8 — Nhiễu do mưa: 5/5 đạt

✔	Mưa mô phỏng giữ đúng trung bình	5.004 mm/h
✔	Phân phối mũ: độ lệch chuẩn = trung bình	tỉ số 0.9998
✔	Không mưa thì không thêm nhiễu	
✔	Mưa càng to nhiễu càng lớn	20 GHz
✔	Tần số càng cao càng chịu mưa	ở mưa 3.3 mm/h

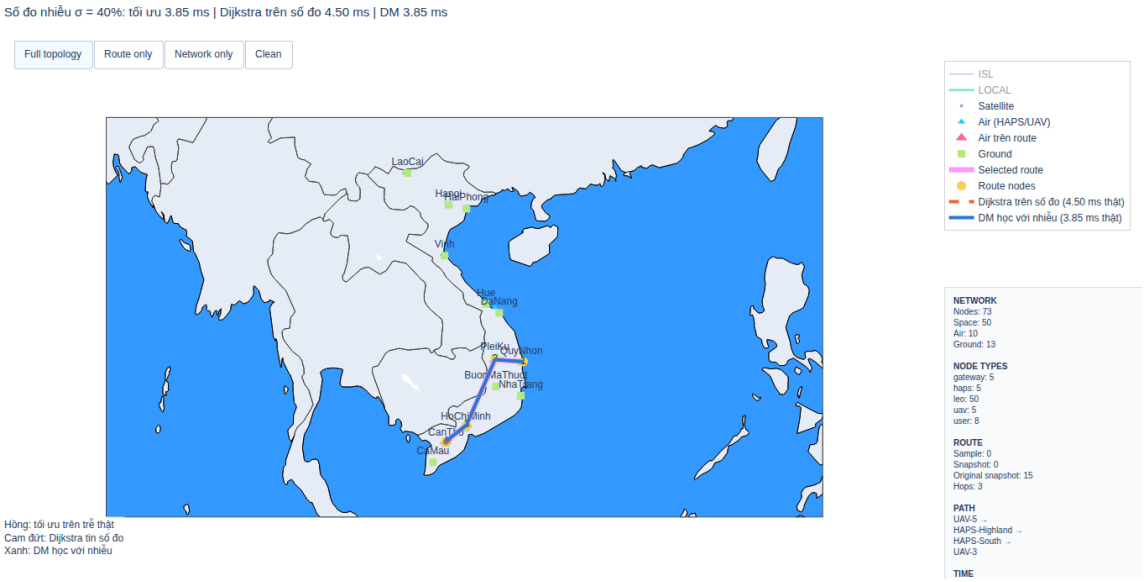
15. Bước 9 — Bản đồ tuyến trên lãnh thổ Việt Nam

Tệp chạy: `huong_dan/buoc_9_ban_do.py` · **Tệp dự án:** `visualization.py` (bản đồ của nhóm), `truc_quan_dm.py` (viết tiếp) · **Dữ liệu:** `data/tle/starlink.txt`, `data/topojson/world_50m.json`

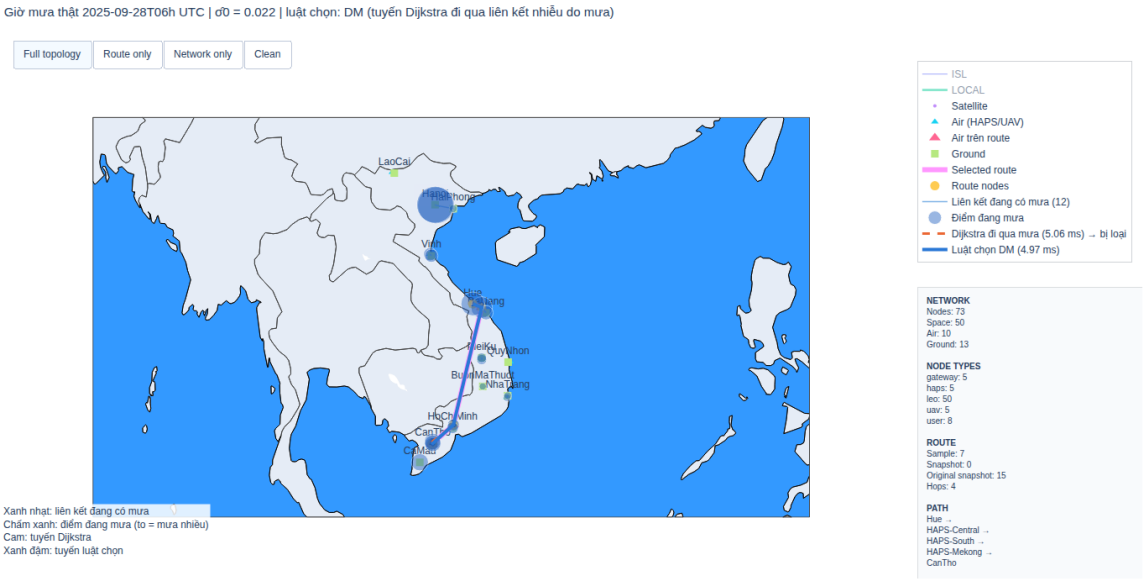
Bước cuối vẽ tuyến lên bản đồ thật. `truc_quan_dm.py` dùng lại hàm `build_figure` của `visualization.py`: vị trí vệ tinh tính lại từ TLE đúng thời điểm của snapshot, vị trí HAPS / UAV / trạm lấy từ `src/physics`. Phần viết thêm vẽ tuyến DM, tuyến Dijkstra, liên kết có mưa và điểm đang mưa, rồi nhúng sẵn bản đồ nền để mở được khi không có mạng. Bước này mở ba trang trong trình duyệt (kéo để di chuyển, cuộn để phóng to, rê chuột lên nút để xem tên); các tệp nằm trong `huong_dan/ket_qua/`.



Bản đồ 1, dữ liệu sạch: tuyến DM trùng tuyến tối ưu (8,59 ms).



Bản đồ 2, số đo nhiễu $\sigma = 40\%$: Dijkstra trên số đo đi tuyến 4,50 ms thật, DM đi đúng tuyến tối ưu 3,85 ms.



Bản đồ 3, một giờ mưa thật (6 giờ UTC ngày 28/9/2025): chấm xanh là điểm đang mưa (to = mưa nhiều), nét xanh nhạt là liên kết có mưa. Tuyến Dijkstra đi qua liên kết nhiễu do mưa nên luật chọn DM (4,97 ms so với 5,06 ms).

Bước 9 — Bản đồ tuyến: 3/3 đạt

✔	Bản đồ chế độ 'sach'	DM so với tối ưu — tối ưu 8.59 ms, DM 8.59 ms (trùng nhau)
✔	Bản đồ chế độ 'nhieu'	Số đo nhiễu σ = 40%: tối ưu 3.85 ms | Dijkstra trên số đo 4.50 ms | DM 3.85 ms
✔	Bản đồ chế độ 'mua'	Giờ mưa thật 2025-09-28T06h UTC | σ0 = 0.022 | luật chọn: DM (tuyến Dijkstra đi qua liên kế…

Muốn vẽ truy vấn khác, chạy `truc_quan_dm.py` với tham số (**Run → Edit Configurations → Parameters**), ví dụ `--che-do nhieu --sigma 0.6 --sample 120`; `--che-do` nhận `sach`, `nhieu`, `mua`.

16. Chạy tất cả và trang kết quả

Tệp chạy: `huong_dan/chay_tat_ca.py` · **Cấu hình:** *Chạy tất cả (trang kết quả)* · **Thời gian:** 1–6 phút

Tệp này chạy lần lượt mười bước mà không mở cửa sổ hình nào, lưu mọi hình vào `huong_dan/ket_qua/`, và mở ngay trang `bang_dieu_khien.html` trong trình duyệt. Trang tự làm mới 5 giây một lần trong lúc chạy: ô của bước đang chạy có viền vàng, bước xong viền xanh (đạt hết) hoặc đỏ (có lỗi). Bấm vào một ô để tới phần của bước đó, gồm mô tả, các hình và bảng kiểm tra. Ở bước 9 có nút mở ba bản đồ. Muốn chạy vài bước, thêm tham số `--buoc 3 5 7` trong **Edit Configurations → Parameters**.

DM + GNN định tuyến SAGSIN — kết quả chạy trên máy bạn

Xong: 97/97 kiểm tra đạt, 46 giây. Mỗi ô là một tệp huong_dan/buoc_*.py; bấm vào ô để tới phần của bước đó.

Bước 0
Kiểm tra máy
17/17 đạt · 2 s

Bước 1
Dữ liệu SAGSIN của TV1
4/4 đạt · 1 s

Bước 2
Chạy các file test của dự án
49/49 đạt · 12 s

Bước 3
Mô hình GNN + DM định tuyến
4/4 đạt · 2 s

Bước 4
Bốn quy mô mạng: độ chính xác và thời gian
4/4 đạt · 7 s

Bước 5
Liên kết bị hỏng
4/4 đạt · 2 s

Bước 6
So với Dijkstra khi số đo có nhiễu
3/3 đạt · 9 s

Bước 7
Luật chia đôi Dijkstra / DM với thời tiết thật
4/4 đạt · 7 s

Bước 8
Nhiễu do mưa và giông, suy từ vật lý
5/5 đạt · 1 s

Bước 9
Bản đồ tuyến trên lãnh thổ Việt Nam
3/3 đạt · 4 s

Bước 0 — Kiểm tra máy

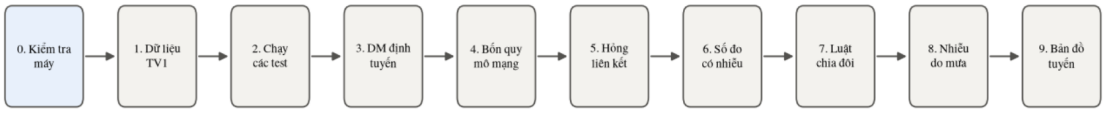
17/17 đạt · 2 s

Kiểm tra phiên bản Python, các thư viện cần có và các tệp dữ liệu, mô hình của dự án.

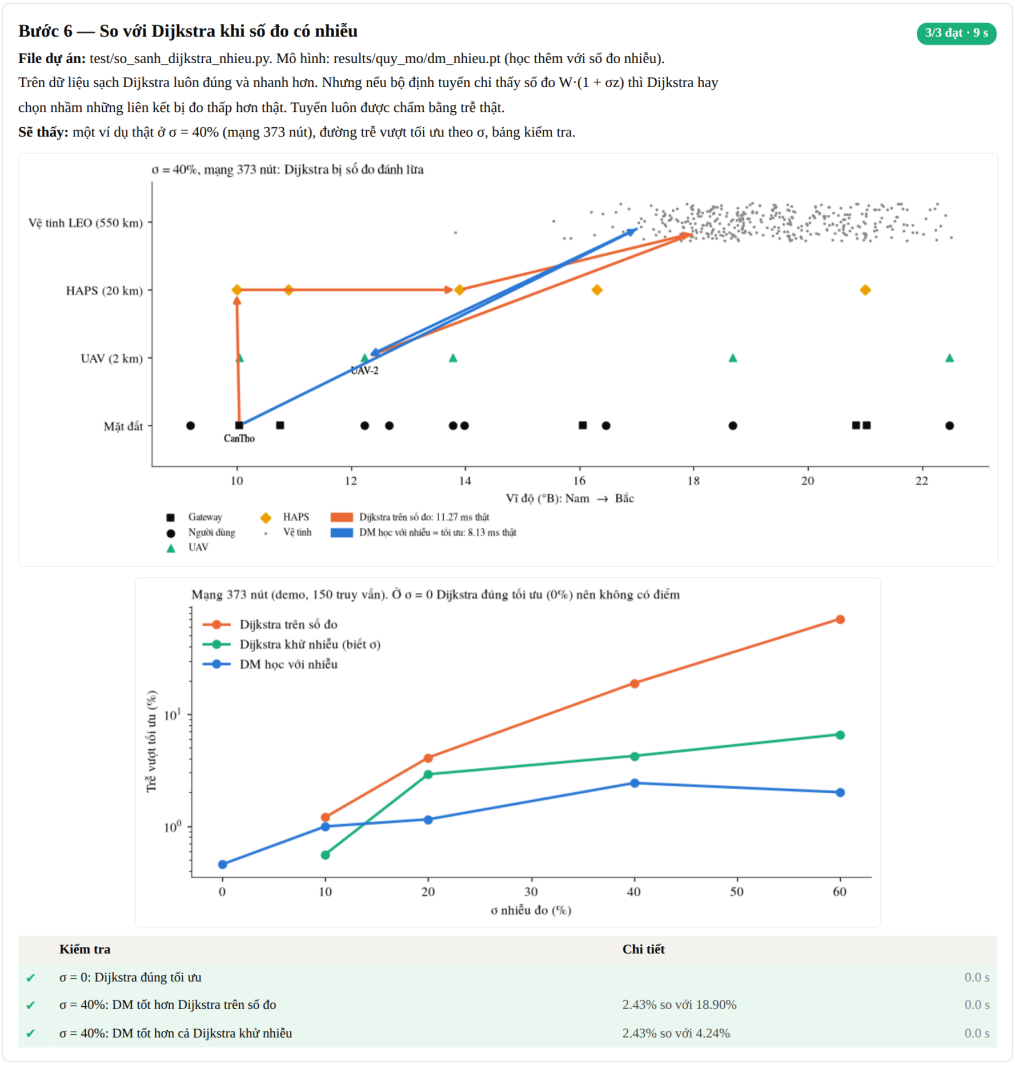
Chạy trong PyCharm: bấm ▶ xanh cạnh dòng `if __name__ == "__main__":` (hoặc chuột phải vào tệp → Run).

Sẽ thấy: sơ đồ 10 bước của hướng dẫn và bảng kiểm tra xanh / đỏ.

Các bước của hướng dẫn: mỗi bước là một tệp huong_dan/buoc_*.py



	Kiểm tra	Chi tiết	
✔	Python ≥ 3.10	3.13.13	0.0 s
✔	Thư viện numpy	phiên bản 2.5.3 — dùng cho mọi bước	0.0 s
✔	Thư viện scipy	phiên bản 1.18.1 — dùng cho mọi bước	0.0 s
✔	Thư viện torch	phiên bản 2.6.0+cu124 — dùng cho mô hình DM	1.1 s
✔	Thư viện matplotlib	phiên bản 3.11.2 — dùng cho vẽ hình	0.0 s

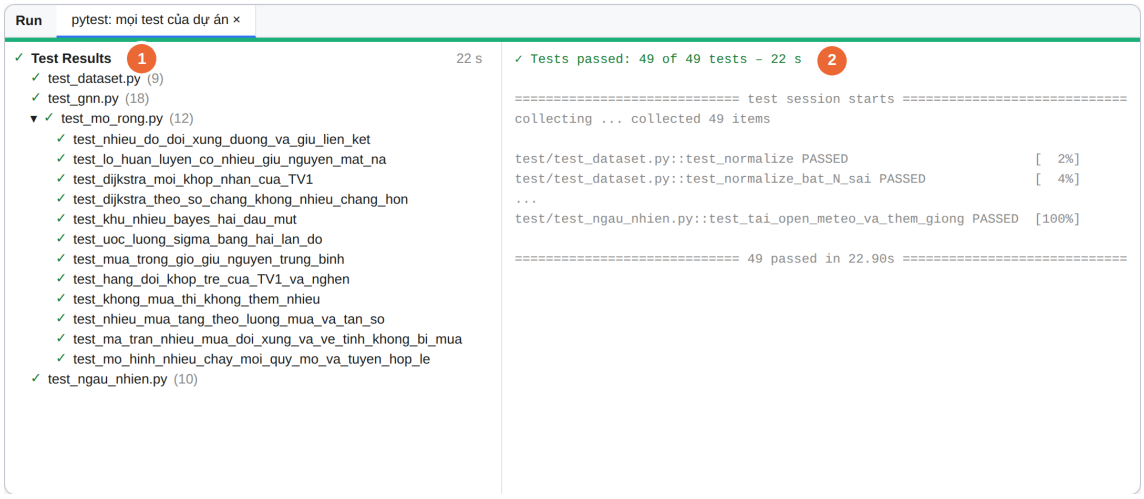


Hai hình trên là ảnh chụp thật của trang kết quả: phần đầu trang và phần của bước 6.

17. Chạy test bằng pytest trong PyCharm

Nhánh mới có tệp `pytest.ini` ở thư mục gốc, nói cho pytest chỉ tìm test trong `test/test_*.py`. Nếu thiếu tệp này, pytest sẽ gặp `test/mini_test.py` của nhóm (tên kết thúc bằng `_test.py`) và báo lỗi vì tệp đó cần `checkpoints_diffusion.pt`. Ba cách chạy:

- Cấu hình *pytest: mọi test của dự án* trong ô chọn cấu hình (Hình 4), bấm ▶.
- Chuột phải thư mục `test` → **Run 'pytest in test'**. Nếu menu ghi *Unittests* thay vì *pytest*: **Settings → Tools → Python Integrated Tools → Default test runner: pytest**.
- Mở một tệp test, bấm ▶ cạnh một hàm `def test_...` để chạy riêng hàm đó.



Hình 6. Cửa sổ Run của pytest (hình giản lược, tên test và số liệu là thật): ① cây test, mỗi tệp một nhánh, dấu ✔ xanh là đạt; ② dòng tổng kết. Test đỏ có dấu ✘; bấm vào nó, khung bên phải hiện dòng lệnh gây lỗi và giá trị so sánh.

18. Các tệp thí nghiệm đầy đủ (không bắt buộc)

Mười bước trên dùng dữ liệu demo nhỏ để chạy trong vài phút. Các con số trong báo cáo được tính bằng những tệp dưới đây trên dữ liệu đầy đủ. Chúng chạy được trong PyCharm theo cách 1 hoặc 2 ở mục 5, tham số đặt ở **Edit Configurations → Parameters**; phần lớn in kết quả ra tệp JSON trong `results/`.

Tệp	Việc làm
`test/so_sanh_dijkstra_nhieu.py`	DM so với năm biến thể Dijkstra ở σ từ 0 đến 60%, bốn quy mô (bảng lớn trong báo cáo).
`test/luat_chia_doi.py`	Luật chia đôi trên nhiều giờ thời tiết; `--mua vat_ly` dùng Δ từ vật lý.
`test/phan_tich_nhieu_mua.py`	Bảng Δ theo loại liên kết và cường độ mưa.
`test/danh_gia_quy_mo.py`	Độ chính xác, thời gian theo quy mô, hỏng liên kết.
`src/ai/train_da_quy_mo.py`	Huấn luyện lại mô hình đa quy mô / học với nhiễu (vài giờ trên CPU).
`test/chay_quy_mo.sh`	Sinh dữ liệu 173–373 nút bằng bộ sinh của TV1, huấn luyện và đánh giá (6–7 giờ; chạy bằng Git Bash).

Dữ liệu đầy đủ 173–373 nút (thư mục `data/quy_mo/`) không có trong nhánh vì nặng hàng trăm MB; `test/chay_quy_mo.sh` sinh lại được từ bộ sinh dữ liệu của TV1.

19. Lỗi thường gặp

Hiện tượng	Cách xử lý
`ModuleNotFoundError: No module named 'torch'` (hoặc plotly, skyfield)	Interpreter của dự án chưa có thư viện: kiểm tra Hình 2, rồi cài lại ở mục 4 trong Terminal có `(.venv)`.

Hiện tượng	Cách xử lý
`No module named 'src'` hoặc `'huong_dan'`	Tệp được chạy từ bản chép ở chỗ khác. Chạy đúng tệp trong thư mục dự án; các tệp bước tự thêm thư mục gốc vào đường dẫn.
Bước 0 báo thiếu tệp `results/...pt`	Đang ở nhánh `master`. Chuyển sang `tv2-dm-gnn` (mục 2).
Không thấy nhóm cấu hình *Hướng dẫn PBL4*	Kiểm tra thư mục `.run` có trong dự án; **File → Invalidate Caches** hoặc mở lại dự án; hoặc chạy bằng chuột phải → Run.
Hình mở thành nhiều cửa sổ, chạy không kết thúc	Không có tab Plots nên mỗi hình là một cửa sổ; đóng hết cửa sổ hình. Hoặc bật **Settings → Tools → Python Plots → Show plots in tool window**.
Bản đồ không tự mở	Mở tay `huong_dan/ket_qua/ban_do_*.html` (chuột phải → **Open In → Browser**).
Chữ trong hình bị ô vuông	Máy thiếu phông Times New Roman (hiếm trên Windows); hình vẫn đúng, matplotlib tự dùng phông thay thế.
Thời gian đo khác tài liệu	Bình thường: thời gian phụ thuộc CPU và việc máy đang làm gì khác. Kiểm tra không xét thời gian.
`git fetch` báo *not a git repository*	Terminal không đứng ở thư mục repo; gõ `cd` tới thư mục PBL4 rồi chạy lại.
`git fetch` báo *Repository lacks these prerequisite commits*	Repo chưa có commit `c9faa1e`: chạy `git pull origin master` rồi lấy lại nhánh.
Git báo `dataset/__pycache__/dataset.cpython-313.pyc` bị thay đổi	Tệp biên dịch tự sinh này đã có sẵn trên `master` (lỡ đưa lên git) nên Python ghi đè mỗi lần chạy. Bỏ qua, đừng commit; hoặc hoàn lại bằng `git checkout -- dataset/__pycache__`.

20. Sau khi chạy xong: đưa lên repo nhóm

Nhánh `tv2-dm-gnn` mới chỉ nằm trên máy bạn. Khi bạn và nhóm đồng ý với kết quả, đẩy nhánh (không đẩy vào `master`):

```
git push -u origin tv2-dm-gnn
```

rồi mở Pull Request từ `tv2-dm-gnn` vào `master` trên GitHub để TV1 và TV3 xem trước khi gộp. Trong PyCharm, cũng làm được bằng **Git → Push** (chọn đúng nhánh `tv2-dm-gnn`). Thư mục `huong_dan/ket_qua/` đã nằm trong `.gitignore` nên kết quả chạy trên máy không bị đẩy lên.